



Sílabo de curso "Ingeniería de Software I"

1. Datos generales

- | | | | |
|-----------------------------------|--------------------------------------|--------------------------------|--|
| 1.1. Programa de estudio: | Ingeniería en Informática y Sistemas | 1.9. Duración semestre: | 18 semanas (18/08/25-29/12/25) |
| 1.2. Área curricular: | Estudios de especialidad | 1.10. Horas: | 06 Hrs (02 HT, 02 HP, 02 HL) |
| 1.3. Dpto. académico: | Ingeniería en Informática y Sistemas | 1.11. Créditos: | 04 créditos |
| 1.4. Código: | IIS. 19.06432 | 1.12. Prerrequisito: | Diseño de Sistemas |
| 1.5. Currículo: | 2018 - F2 | 1.13. Docente: | Ing. Gianfranco A. Málaga Tejada, Mag.
gmalagat@unibg.edu.pe |
| 1.6. Año académico: | 2025-II | | Ing. Manuel Yuri Apaza Valencia, Mtro.
mapazav@unibg.edu.pe |
| 1.7. Semestre de estudios: | Sexto ciclo | | |
| 1.8. Modalidad: | Presencial | | |

2. Sumilla

2.1. Naturaleza

La asignatura corresponde al área de formación especializada, es de carácter teórico-práctica.

2.2. Propósito

Tiene como propósito brindar los fundamentos teóricos y prácticos que rigen los procesos de ingeniería y construcción del software. Dotar a los estudiantes con habilidades para definir, evaluar y mejorar procesos de ingeniería del software, así como comprender las técnicas y herramientas esenciales para la construcción eficiente de soluciones software. El énfasis recae en la medición, gestión y tecnologías aplicables.

2.3. Contenidos básicos

Proceso de ingeniería de software: Definición de procesos de ingeniería de software, ciclos de vida del software, evaluación y mejora del proceso del software, medición del software, herramientas de proceso de ingeniería del software. Construcción del software: Fundamentos de la construcción del software, gestión de la construcción, consideraciones prácticas, tecnologías para la construcción, herramientas de construcción de software.

3. Competencias

3.1. Competencia Genérica Institucional:

Desarrolla pensamiento crítico, muestra habilidades como investigador, innovador y creativo. Demuestra habilidad para realizar trabajo en equipo, se comunica de manera eficaz en la propia profesión, toma decisiones, es hábil en la gestión de información y manejo de las TIC y resuelve problemas.

3.2. Competencias del perfil de egreso:

- Planificar, liderar y ejecutar proyectos de desarrollo software bajo estándares de calidad, utilizando herramientas de ingeniería para satisfacer las necesidades del entorno de su actividad profesional, proponiendo mejoras en herramientas tecnológicas y procesos de la organización.
- Tener la capacidad de investigación tecnológica conociendo las teorías de la información y trabajo en equipos multidisciplinarios para búsqueda de nuevos conocimientos.

4. Organización de los aprendizajes

1ª Unidad: Proceso de la Ingeniería de Software

Competencias genéricas. Desarrolla el **pensamiento crítico** y la **resolución de problemas** al analizar y comparar diversos modelos de procesos para determinar el más adecuado según el contexto de un proyecto. Fomenta el **trabajo en equipo** y la **comunicación efectiva** a través de la discusión de casos de estudio y la planificación colaborativa de las actividades de análisis de negocio y gestión de requisitos.

Competencias específicas. Aplica los principios y fundamentos de la ingeniería de software para gestionar el ciclo de vida de un proyecto de desarrollo. **Diseña y gestiona procesos de software**, seleccionando los modelos y métodos más apropiados para garantizar la calidad del producto final y la satisfacción de las necesidades de los interesados, en línea con estándares.

Resultados de aprendizaje del estudiante. **Explica** los principios de la ingeniería de software, **compara** los diferentes modelos de ciclo de vida del software y **aplica** técnicas para la planificación del análisis de negocio y la gestión del ciclo de vida de los requisitos, asegurando su correcta priorización, trazabilidad y mantenimiento en un proyecto de software.

Evidencia: A través de la **resolución de casos prácticos** donde se aplican modelos de proceso y técnicas de gestión de requisitos. Se complementa con la **presentación y defensa de un trabajo de investigación** sobre modelos de procesos emergentes y una **evaluación escrita** que mide la comprensión conceptual de los temas de la unidad.

Semana%	Contenidos	Actividades	Estrategias metodológicas
1 6%	Introducción a la Ingeniería de Software. Definición de la disciplina de Ingeniería de Software, su alcance y objetivos. La ética y la práctica profesional en el desarrollo de software. El impacto económico y social del software.	Discusión de casos de estudio sobre el impacto de la ingeniería de software en proyectos reales y la sociedad. Identificación de los principios éticos del código de la ACM e IEEE en escenarios prácticos de desarrollo de software.	Valora la disciplina de la ingeniería de software y se compromete con la práctica ética y profesional.





2 12%	Fundamentos del Proceso de Software. Definición de proceso de software, actividades, y tareas. Introducción a los modelos de ciclo de vida del software, sus categorías y la razón fundamental de su utilización en proyectos.	Análisis comparativo de diferentes modelos de ciclo de vida del software aplicando sus características a un caso de estudio. Elaboración de un diagrama que ilustre las fases y actividades principales de un modelo de ciclo de vida seleccionado.	Aplica un pensamiento orientado a procesos y demuestra adaptabilidad en la selección de metodologías.
3 18%	Modelos de Proceso de Software. Modelos prescriptivos como el modelo en cascada y el modelo en V. Modelos de proceso evolutivo como el prototipado y el modelo espiral. El Proceso Unificado y sus fases.	Aplicación de un modelo de proceso prescriptivo para planificar un proyecto pequeño. Desarrollo de un prototipo de baja fidelidad para ilustrar un ciclo de proceso evolutivo y obtener retroalimentación temprana.	Valora la planificación de los modelos prescriptivos y la gestión de riesgos de los modelos evolutivos.
4 24%	Métodos Ágiles de Desarrollo de Software. Principios y valores del Manifiesto Ágil. Introducción a marcos de trabajo ágiles como Scrum y Extreme Programming (XP). Roles, artefactos y eventos en Scrum.	Simulación de un ciclo de Scrum (Sprint), incluyendo la planificación, la reunión diaria y la retrospectiva. Elaboración de un Product Backlog y un Sprint Backlog para un caso de estudio simple.	Fomenta la colaboración, la comunicación y la adaptabilidad en entornos de trabajo ágiles.
5 30%	Planificación y Monitoreo del Análisis de Negocio. El Rol del Análisis de Negocio en la Ingeniería de Software. Planificación del enfoque de análisis de negocio, el involucramiento de los interesados, la gobernanza y la gestión de la información.	Elaboración de un plan de análisis de negocio para un proyecto, identificando stakeholders clave y definiendo sus roles y responsabilidades. Aplicación de técnicas para la planificación del involucramiento de los interesados.	Demuestra proactividad en la planificación y valora la comunicación estructurada con los interesados.
6 36%	Gestión del Ciclo de Vida de los Requisitos. Trazabilidad de requisitos. Mantenimiento y reutilización de requisitos. Priorización de requisitos utilizando diversas técnicas.	Creación de una matriz de trazabilidad para vincular requisitos de negocio, de stakeholders y de solución. Aplicación de una técnica formal de priorización, como MoSCoW, a un conjunto de requisitos de un caso práctico.	Aplica el rigor en la gestión del ciclo de vida de los requisitos y en la toma de decisiones de priorización.
7 42%	Fundamentos de la Calidad del Software. Cultura y ética en la calidad. El valor y los costos de la calidad. Estándares y modelos de calidad. La dependencia e integridad del software.	Análisis de los costos de la calidad (prevención, evaluación, fallas internas y externas) en un escenario de desarrollo. Identificación de los atributos de calidad relevantes para un sistema de software específico basado en el modelo ISO/IEC 25010.	Promueve una cultura de calidad y es consciente de su impacto en el éxito del producto y del negocio.
8 48%	Presentación de trabajos de investigación		
9 51%	Evaluación de conocimientos, evaluación de desempeño, evaluación de producto		

2ª Unidad: Construcción del software

Competencias Genéricas: Refuerza la **capacidad de aprendizaje autónomo** y la **adaptación a nuevas tecnologías** al investigar e implementar diversas herramientas y técnicas de construcción de software. Promueve la **atención al detalle** y el **razonamiento lógico** mediante la aplicación de estándares de codificación, prácticas de programación defensiva y depuración de código.

Competencias Específicas: Construye componentes de software robustos, mantenibles y de alta calidad, traduciendo los artefactos del diseño en código funcional. **Aplica estándares de codificación**, gestiona el proceso de construcción y **utiliza tecnologías y herramientas modernas** para asegurar la calidad interna del software, siguiendo las mejores prácticas.

Resultados de aprendizaje del estudiante: Al finalizar la unidad, el estudiante **construye** componentes de software aplicando estándares de codificación y principios de minimización de la complejidad. Además, **gestiona** el proceso de construcción mediante la planificación y el control de dependencias, y **asegura** la calidad del código a través de técnicas de programación defensiva, manejo de excepciones y uso de tecnologías de construcción apropiadas.

Evidencia: Entrega de un **componente de software funcional**, el cual es valorado según su adherencia a estándares de codificación y calidad a través de revisiones de código. Adicionalmente, se considera la **presentación de una investigación** sobre tecnologías de construcción y una **evaluación escrita** que valida el dominio de los conceptos clave de la unidad.

Semana%	Contenidos	Actividades	Estrategias metodológicas
10 57%	Fundamentos de la Construcción del Software. Minimización de la complejidad. Anticipación al cambio y construcción para la verificación. El concepto de reutilización de activos de software. La importancia de los estándares en la construcción.	Análisis de fragmentos de código para identificar y reducir su complejidad ciclométrica. Aplicación de principios de diseño para anticipar cambios, como el uso de interfaces y abstracción.	Valora la simplicidad, legibilidad y mantenibilidad del código como pilares de un trabajo profesional.
11 63%	Gestión de la Construcción. La construcción en los diferentes modelos de ciclo de vida. Planificación y medición de la construcción. Gestión de	Planificación de la secuencia de construcción e integración de componentes para un sistema modular. Uso de herramientas de gestión de	Aplica un enfoque de ingeniería en la planificación de la construcción y asume la





	dependencias internas y externas en el software.	dependencias (como Maven o npm) para un proyecto pequeño.	responsabilidad sobre la gestión de dependencias.
12 69%	Consideraciones Prácticas de la Construcción. Diseño detallado durante la construcción. Lenguajes de construcción, codificación y organización del código fuente. Documentación del código.	Aplicación de un estándar de codificación (layout, convenciones de nombrado) para mejorar la legibilidad del código. Realización de una revisión de código por pares para identificar mejoras en la documentación y estructura.	Demuestra profesionalismo a través de la producción de código limpio y valora la retroalimentación de sus pares.
13 75%	Calidad en la Construcción. Programación defensiva y uso de aserciones. Diseño por contrato. Manejo de errores y excepciones. Técnicas de depuración de código.	Implementación de técnicas de programación defensiva para validar entradas y manejar estados inesperados. Uso de bloques try-catch para una gestión estructurada de excepciones y errores.	Desarrolla una mentalidad proactiva para la prevención de defectos, escribiendo código robusto y seguro.
14 81%	Tecnologías de Construcción I. Diseño y uso de APIs. Aspectos de ejecución en tiempo de ejecución orientados a objetos. Parametrización, plantillas y genéricos.	Diseño de una Interfaz de Programa de Aplicaciones (API) simple, clara y consistente para un módulo de software. Aplicación de genéricos o plantillas para crear componentes de software reutilizables y con seguridad de tipos.	Aprecia el valor de las APIs bien diseñadas y muestra curiosidad por las tecnologías de construcción robustas.
15 87%	Tecnologías de Construcción II. Técnicas de construcción basadas en estado y dirigidas por tablas. Configuración en tiempo de ejecución e internacionalización. Primitivas de concurrencia.	Modelado del comportamiento de un componente de software utilizando una máquina de estados finitos. Implementación de la internacionalización en una interfaz de usuario para soportar múltiples idiomas.	Valora el uso de modelos formales para gestionar la complejidad y reconoce la importancia del software adaptable.
16 93%	Presentar proyectos de desarrollo de software		
17 96%	Seminario de trabajos de investigación		
18 100%	Evaluación de conocimientos, evaluación de desempeño, evaluación de producto		

5. Programa de prácticas

1ª Unidad

Sem.	Tema	Resultado Esperado	Recursos Necesarios
1	Constitución del Proyecto y Equipo.	Informe 1: Acta de Constitución del Proyecto. Incluirá: nombre del equipo, roles asignados (2-3 por estudiante), cliente seleccionado, justificación, objetivos, alcance inicial y stakeholders identificados.	Plantilla de Acta de Constitución. Acceso y comunicación inicial con el cliente real.
2	Selección del Proceso de Software.	Monografía 1: Selección y Justificación de la Metodología. El equipo investiga y compara 2-3 modelos de proceso, seleccionando y justificando uno para su proyecto y cliente específico.	Guías. Artículos académicos sobre metodologías de desarrollo.
3	Planificación Inicial del Proyecto.	Informe 2: Plan de Gestión del Proyecto (versión inicial). Según la metodología elegida, puede ser un Product Backlog priorizado (Scrum) o una Estructura de Desglose de Trabajo - EDT (tradicional).	Herramientas de gestión de proyectos (Jira, Trello, ProjectLibre). Plantillas de Backlog/EDT.
4	Profundización en Métodos Ágiles.	Informe 3: Plan de Iteración/Sprint 1. El equipo detalla el primer ciclo de desarrollo, definiendo el objetivo, las tareas y los criterios de aceptación para un conjunto pequeño de funcionalidades.	Software para gestión ágil. Material sobre Scrum/Kanban.
5	Plan de Involucramiento de Stakeholders.	Informe 4: Plan de Gestión de Interesados. Identifica a todos los stakeholders, su nivel de influencia e interés, y define la estrategia de comunicación y colaboración con ellos (especialmente con el cliente).	Guía. Plantilla de matriz de stakeholders.
6	Gestión y Trazabilidad de Requisitos.	Informe 5: Matriz de Trazabilidad de Requisitos. Vincula los requisitos del negocio (necesidades del cliente) con los requisitos funcionales y no funcionales que se construirán.	Guía. Plantillas de matriz de trazabilidad. Hojas de cálculo (Excel, Google Sheets).
7	Planificación de la Calidad del Software.	Monografía 2: Plan de Aseguramiento de la Calidad (versión inicial). Define los estándares de calidad para el proyecto (ej. métricas de código, estándares de documentación) y las actividades de calidad a realizar.	Estándar ISO/IEC 25010. Guía.

2ª Unidad

Sem.	Tema	Resultado Esperado	Recursos Necesarios
10	Definición de Estándares de Construcción.	Informe 6: Documento de Estándares de Codificación. El equipo define y acuerda las reglas para nombrar variables, formato de código, comentarios y	Guías de estilo de lenguajes de programación (ej. PEP 8 para





		organización de archivos para su lenguaje de programación.	Python, Google Java Style Guide).
11	Configuración del Entorno de Desarrollo.	Informe 7: Evidencia de Configuración del Entorno Colaborativo. Capturas de pantalla y un breve texto que demuestren el repositorio Git/SCM creado, el proyecto base en el IDE y la configuración del gestor de dependencias.	Cuentas en GitHub/GitLab. IDEs (VS Code, Eclipse, IntelliJ). Gestores de dependencias (Maven, npm, pip).
12	Implementación del Baseline Arquitectónico	Commit en Repositorio y Reporte de Revisión por Pares. Se implementa la estructura base del proyecto ("Hola Mundo" con arquitectura). El entregable es el enlace al commit y un informe de la primera revisión de código entre pares.	Sistema de Control de Versiones (Git). Plantilla de informe de Peer Review.
13	Implementación de Calidad: Programación Defensiva.	Commit en Repositorio e Informe 8: Implementación de Feature con Calidad. Desarrollo de una funcionalidad clave aplicando manejo de errores, validaciones de entrada y aserciones. El informe explica las decisiones tomadas.	Repositorio Git. Depurador del IDE.
14	Diseño e Implementación de APIs Internas.	Commit en Repositorio y Monografía 3: Documentación de API. Se diseña y documenta una API interna (ej. para acceder a los datos) utilizando un estándar como OpenAPI/Swagger.	Repositorio Git. Herramientas de documentación de API (Swagger Editor, Postman).
15	Entrega y Presentación del MVP.	Producto Mínimo Viable (MVP) funcional y video de demostración. El equipo entrega el software ejecutable y un video corto (5-7 min) mostrando su funcionalidad principal al cliente.	Software de grabación de pantalla (OBS Studio, Loom). Plataforma para despliegue simple (si aplica).

6. Estrategias metodológicas

Aula Invertida	Es un enfoque diferente de enseñanza donde el estudiante incorpora información antes de la clase y participa posteriormente en ella. No se requiere la presencia física del Docente.
Estrategias para la enseñanza en grupo, centradas en la presentación de información y la colaboración	Parte de la construcción de conocimiento grupal a partir de información suministrada. Intervienen dos roles: el del expositor que puede ser el docente, un experto o un estudiante y, el grupo receptor de la información. Este último tendrá la responsabilidad de realizar actividades en forma individual o grupal que después compartirá a la clase en forma de resultados.
Estrategias centradas en el trabajo colaborativo	Se basa en la construcción de conocimiento en forma grupal empleando estructuras de comunicación de colaboración. Los resultados serán siempre compartidos por el grupo, donde es fundamental la participación activa de todos los miembros de forma cooperativa y abierta hacia el intercambio de ideas del grupo. El docente dará las normas, estructura de la actividad y realizará el seguimiento y la valoración.
Aprendizaje Basado en Problemas (ABP)	Estrategia que motiva el aprendizaje independiente y ejercita a los estudiantes a abstraer un problema o ejercicio dado y encontrar una solución.
Aprendizaje Basado en Proyectos (ABPr)	Estrategia que motiva el aprendizaje grupal al elaborar un proyecto de fin de curso usando los conocimientos adquiridos.
Videoconferencia (o videollamada)	Comunicación simultánea bidireccional de audio y video, que permite mantener reuniones con grupos de personas situadas en lugares alejados entre sí.

7. Sistema de evaluación

7.1. Matriz de evaluación

Unidades	Producto académico	Técnica de evaluación	Instrumento de evaluación
I	Evaluación teórico/práctico virtual o presencial	Observación	Cuestionario
	Asistencia, interés y esfuerzo personal. Trabajo en equipo. Control de lectura. Participación en foro de discusión	Análisis de desempeño	Registro en aula
	Proyecto Académico sobre los temas tratados	Informe/Exposición	Rúbrica
Promedio primer parcial			
II	Evaluación teórico/práctico virtual o presencial	Observación	Cuestionario
	Asistencia, interés y esfuerzo personal. Trabajo en equipo. Control de lectura. Participación en foro de discusión	Análisis de desempeño	Registro en aula
	Proyecto Académico sobre los temas tratados	Informe/Exposición	Rúbrica
Promedio segundo parcial			

7.2. Promedios

Primer parcial	Segundo parcial	Promedio final
----------------	-----------------	----------------





$PP1 = 30\%PEC + 25\%PED + 45\%PEP$	$PP2 = 30\%PEC + 25\%PED + 45\%PEP$	$PF = \frac{PP1 + PP2}{2}$
Donde: PP1 : Promedio Parcial 1 PEC : Promedio evidencias de conocimientos PED : Promedio evidencias desempeño PEP : Promedio evidencias de producto	Donde: PP2 : Promedio Parcial 2 PEC : Promedio evidencias de conocimientos PED : Promedio evidencias desempeño PEP : Promedio evidencias de producto	Donde: PF : Promedio final PP1 : Promedio Parcial 1 PP2 : Promedio Parcial 2

7.3. Consideraciones

- Con los 2 promedios parciales se obtiene el promedio final del curso.
- Antes de la finalización del semestre académico se dará al estudiante la oportunidad de **un examen sustitutorio**, que reemplazará a la nota menor de uno de los promedios parciales.
- Las características del examen sustitutorio se precisarán después de la segunda evaluación (teórico, práctico, mixto, presencial, virtual, etc.)
- El promedio final aprobatorio mínimo será de 11.
- La asistencia inferior al 70% de clases teóricas o al 90% de clases prácticas, en cada unidad, resultará en consignar la nota de cero (00), en la correspondiente unidad. (art. 108º del [Reglamento Académico](#))
- Evaluación permanente.

Acepciones:

- Las sesiones de teoría y práctica se realizan de forma presencial o virtual, síncrona o asíncrona, según la naturaleza de la sesión, el curso o la **normativa vigente**.
- El aula virtual tiene herramientas básicas (tareas, evaluaciones, foros, recursos de aprendizajes, *chats*, calificaciones, grupos y lecturas) y todo lo necesario para el soporte del dictado de clases.

8. Bibliografía

8.1. Bibliografía básica

Nº	Código ubicación en biblioteca	Autor	Título	Año
01	001.25 K41	Kendall, Kenneth & Kendall, Julie	"Análisis y diseño de sistemas"	2011
02	005.3 P85 2002	Pressman, Roger	"Ingeniería del Software. Un Enfoque Práctico"	2002
03		Senn, James	"Análisis y Diseño de Sistemas de Información"	1993
04	005.1 S67 2011	Sommerville, Ian	"Ingeniería de Software"	2011

8.2. Bibliografía complementaria

Se compartirá con el alumnado el material complementario que sea relevante para el dictado del curso, usando la plataforma virtual pertinente.

Abril, 2025

